

SUBIECT engleză

① Which SOLID principle is not followed correctly in the following code snippet?

```
public class A {  
    void hello() { // does something here ... }  
}  
public class B extends A {  
    int i;  
    void hello() { i++; }  
}
```

LSP, DIP

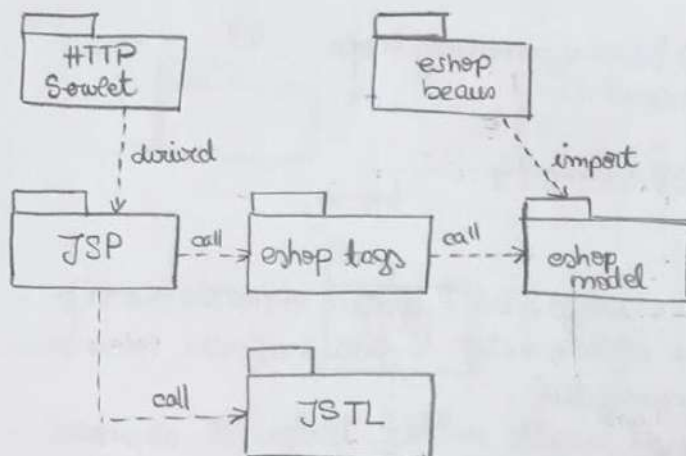
LSP - dacă B este un subtip al lui A, atunci obiectele de tip A pot fi substituite cu obiecte de tip B fără a afecta micuina din proprietățile programului.

DIP - modelele de nivel înalt nu ar trebui să depindă de modulele de nivel scăzut. Ambele ar trebui să depindă de abstracții.

O alternativă ar fi să declarăm o interfață în care să definim metoda hello(). Clasele A și B vor implementa interfața.

```
public interface I {  
    void hello();  
}  
public class A implements I {  
    void hello() { // ... }  
}  
public class B implements I {  
    int i;  
    void hello() { i++; }  
}
```

② Consider the Package Design principle you know, how would you evaluate the following design? Justify. Provide a better design if the case.



Pentru arhitectura din imagine ne respectă Acyclic Dependencies Principle, $\nexists$ cicluri în graful dependențelor.
Pentru a evalua arhitectura calculăm instabilitatea pt. fiecare pachet.

În general, $I = \frac{C_i}{C_i + C_o}$ — outgoing dependencies
incoming dependencies.

$$I_{\text{HTTP}} = \frac{1}{1} = 1 \Rightarrow \text{instabil}$$

$$I_{\text{JSP}} = \frac{2}{3} = 0,66$$

$$I_{\text{JSSTL}} = \frac{0}{1} = 0 \Rightarrow \text{f. stabil}$$

$$I_{\text{eshop tags}} = \frac{1}{2} = 0,5$$

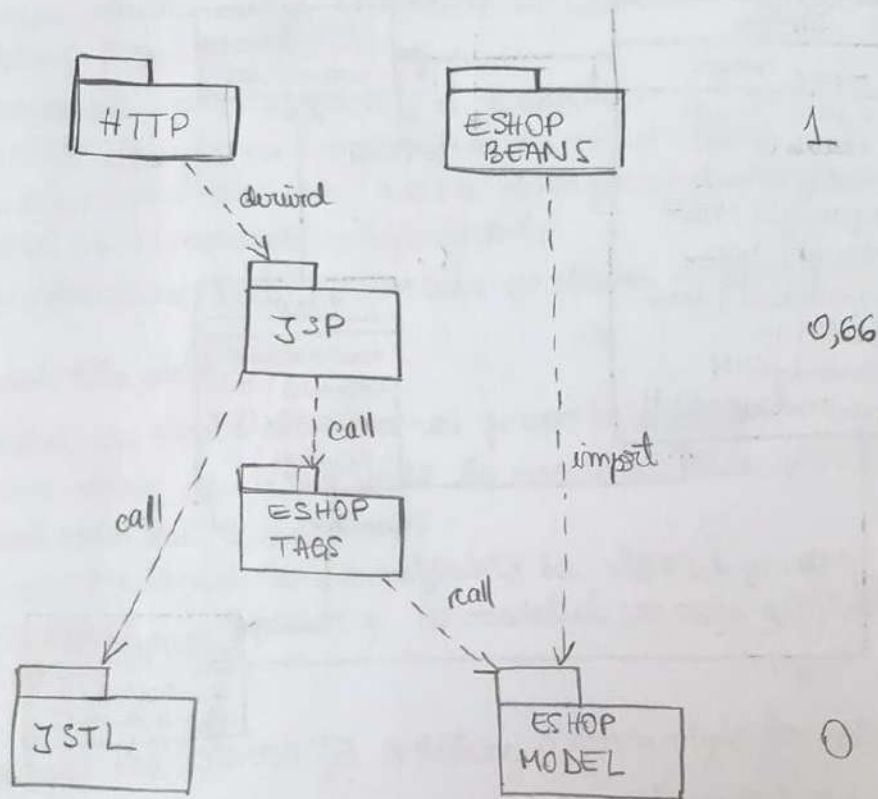
$$I_{\text{eshop model}} = \frac{0}{2} = 0 \Rightarrow \text{f. stabil}$$

$$I_{\text{eshop beans}} = \frac{1}{1} = 1 \Rightarrow \text{instabil}$$

Se observă că avem 2 pachete foarte instabile: HTTPServlet și eshop beans, 2 foarte stabile: eshop model, JSSTL și 2

afiate între stabilitate și instabilitate.

Pachetele stabile trebuie să fie abstracte și să afe la baza design-ului, iar pachetele flexibile trebuie să se afe en top of the design. Aceste două atii se respectă în acest design și este ilustrat acest lucru în figura următoare :

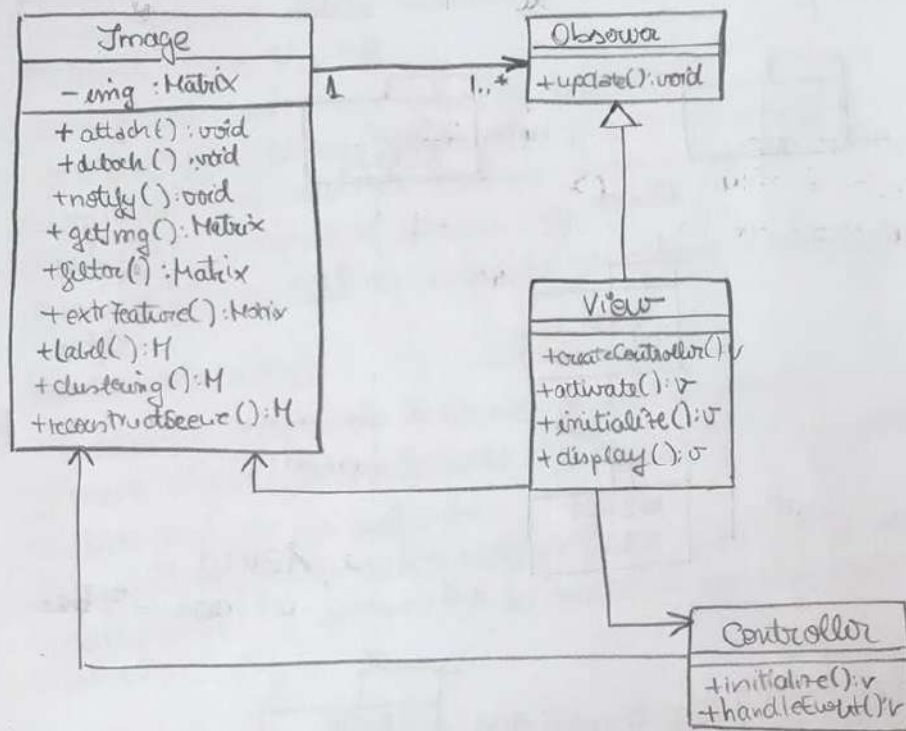


- ③ i) Image recognition system :
- filtering
 - feature extraction
 - labeling
 - clustering
 - scene reconstruction
 - + no fixed order

The system should be easy to update with new algorithms.

i). Model view Controller Pattern.

ii). Give a sketch of sample architecture for the given system. Explain the interaction between the parts of the architecture



The Model (Image)

- Încapsulează și manipulează datele de domeniu care vor fi redată.
- Modelul nu are idee cum să afișeze informațiile pe care le are, nici nu interacționează cu ierarhia sau nu primește nicio intrare de utilizator.
- Încapsulează funcționalitatea necesară manipulării, obținerii și furnizării a unor date mai departe, independent de orice interfață de utilizator sau orice dispozitiv de intrare utilizator.

The View

- este o redare vizuală specifică a informațiilor conținute în model.
- instanțările multiple pot prezenta mai multe extinderi ale datelor din model.

- fiecare View este dependent de un Model
- când se modifică modelul, toate vizualizările dependente sunt actualizate.

The Controller

- manipulează datele introduse de utilizator. Handle requests utilizând Model și View.
- urmărirea cu mouse-ului și tastaturii
- permite decuplarea modelului de la view-ului, permițând view-ului să aibă date pur și simplu și modelului să încapere simplu datele
- Comportamentul c. depinde de starea modelului

How does this work?

- Modelul are o listă de View-uri pe care le suportă.
- Fiecare View are o referință la modelul său, precum și la controlorul său de susținere.
- Fiecare controller are o referință la View-ul pe care îl controlează, precum și la modelul pe care este bazat View-ul.

iii). Discuss the reason for selecting a given style for the given system.

Discuss possible disadvantages.

- crește complexitatea
- posibilitatea unui nr. excesiv de actualizări.
- comunicare intimă între VIEW și CONTROLLER.
- cuplate modelul de view și controller.
- ineficiența accesului de date în VIEW.
- inevitabilitatea schimbării la view și controller la portare.

(B: multiple views for a given model;

Synchronised views;

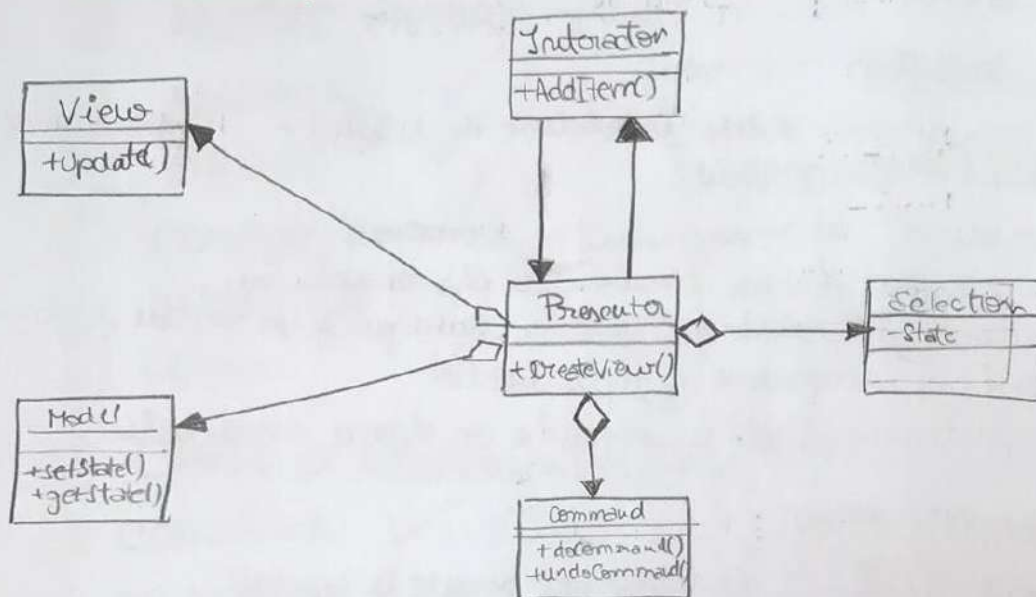
"pluggable" views and controllers

exchangeability of "look and feel"

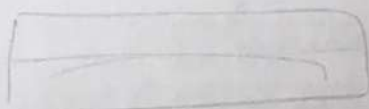
framework potential.

① TAPPOV METHOD ✓

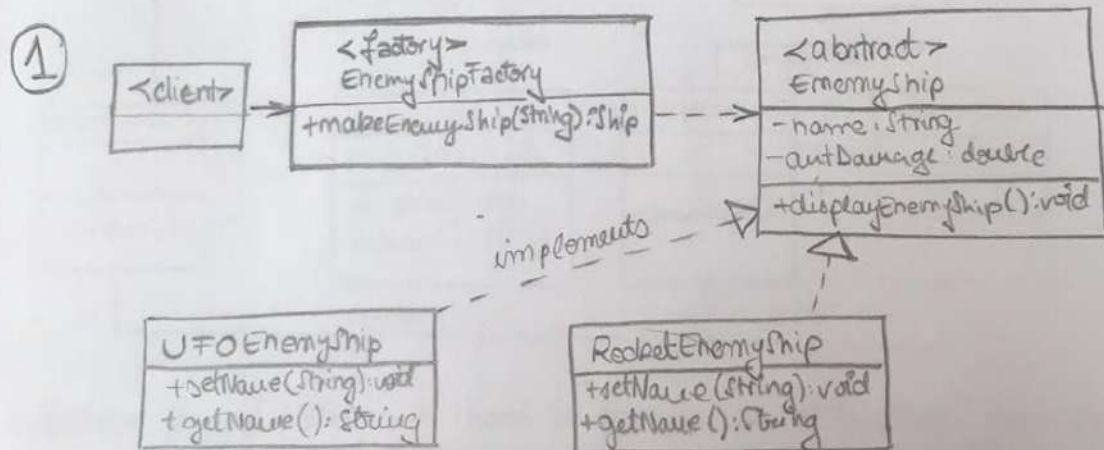
MVP



Presenter

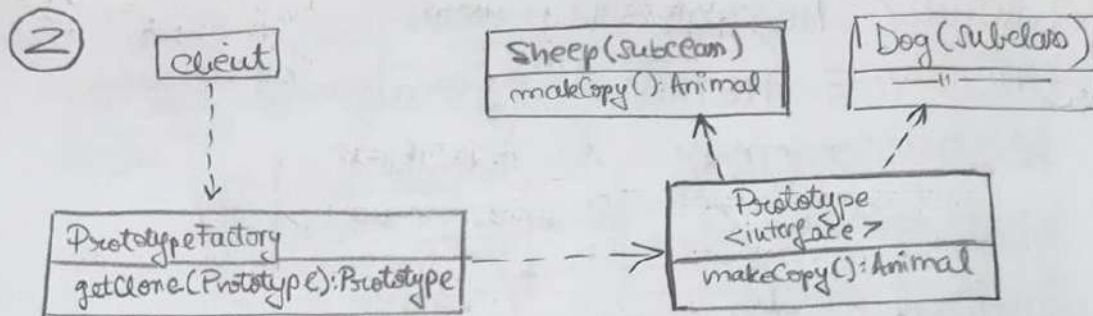


- ① FACTORY METHOD ✓
 - ② PROTOTYPE METHOD X¹
 - ③ ABSTRACT FACTORY X
 - ④ ADAPTER ✓
 - ⑤ BRIDGE ✓
 - ⑥ COMPOSITE ✓
 - ⑦ DELEGATOR ✓
 - ⑧ PROXY ✓
 - ⑨ CHAIN OF RESPONSIBILITY ✓
 - ⑩ COMMAND ✓
 - ⑪ OBSERVER ✓
 - ⑫ STATE
 - ⑬ STRATEGY ✓
- Creational Patterns: ①, ②, ③
- Structural Patterns: ④, ⑤, ⑥, ⑦, ⑧
- Behavioral Patterns: ⑨, ⑩, ⑪, ⑫, ⑬



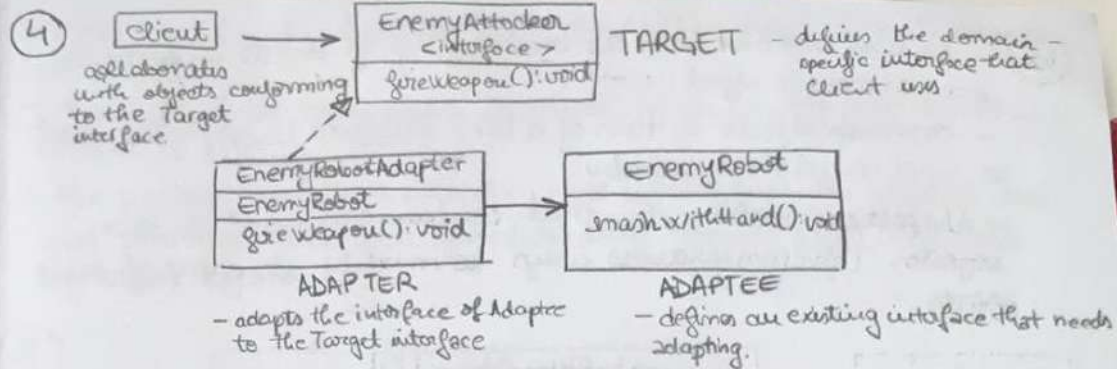
The FM allows you to create obj without specifying the exact class of obj that will be created.

- when a method returns one of several possible classes that share a common super class.
- The class is chosen at runtime
- when you don't know ahead of time what class object you need.
- when all of the potential classes are in the same subclasses hierarchy.
- to centralize class selection code
- when you don't want the user to have to know every sub-class
- to encapsulate object creation

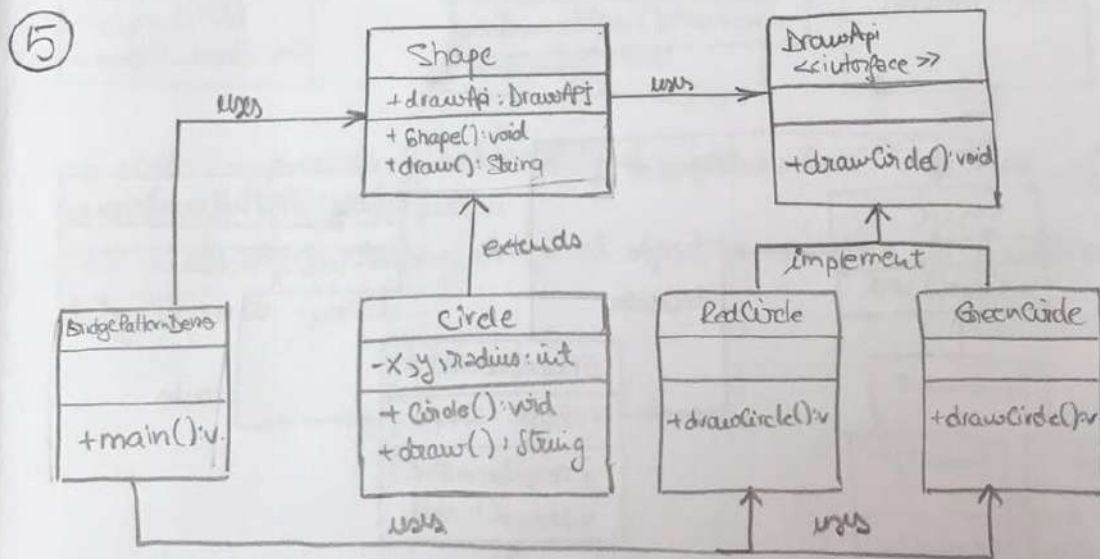


- creating new objects by cloning (copying) other objects
- allows for adding of any subclass instance of a known superclass at runtime.
- when there are numerous potential classes that you want to only use if needed at runtime
- reduces the need for creating subclasses.

③

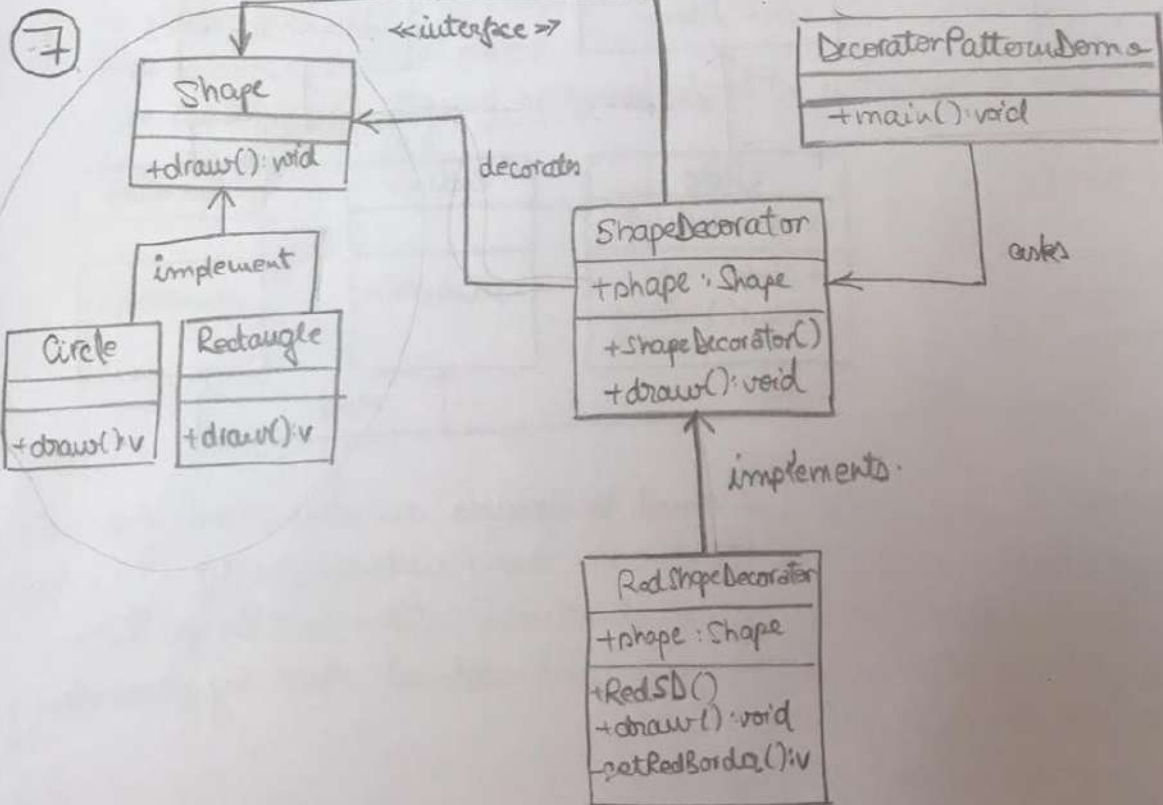
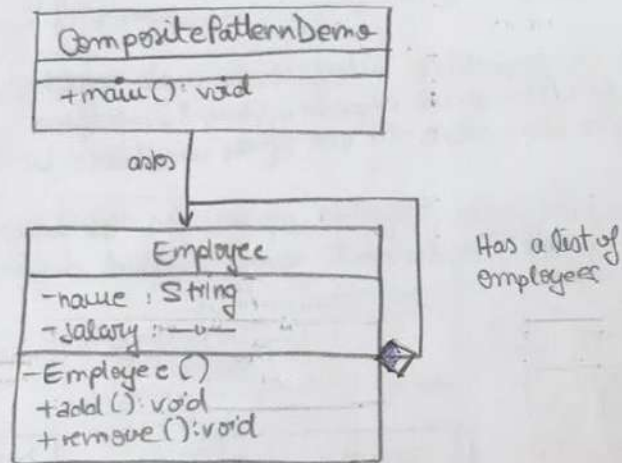


- allows 2 incompatible interfaces to work together
- used when the client expects a (target) interface
- the Adapter class allows the use of the available interface and the Target interface
- any class can work together as long as the Adapter solves the issue that all classes must implement every method defined by the shared interface.

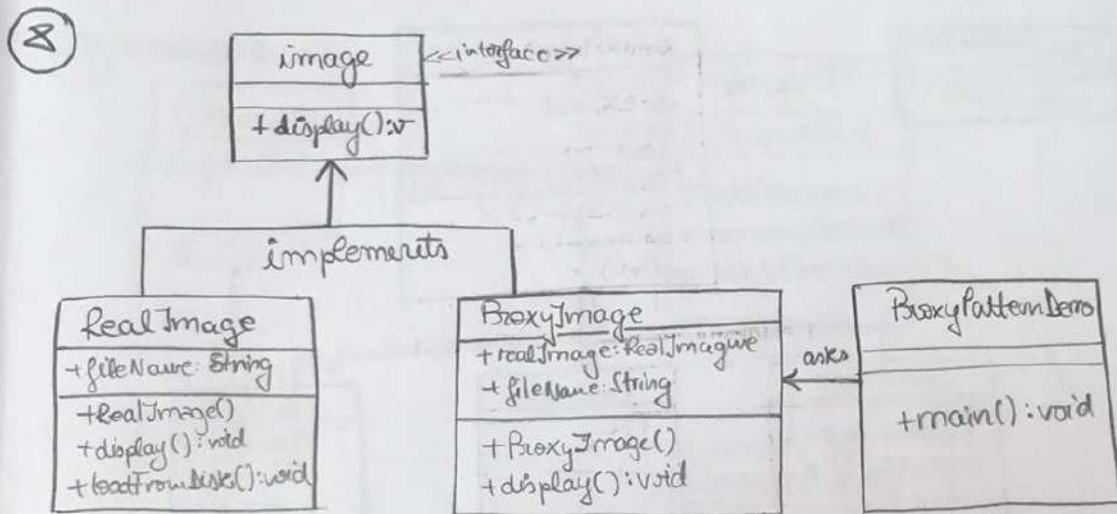


- Bridge is used when we need to decouple an abstraction from its implementation so that the 2 can vary independently. This type of design pattern comes under structural pattern as this pattern decouples implementation class and abstract class by providing a bridge structure between them.
- The Bridge is an interface

- ⑥ - is used where we need to treat a group of objects in similar way as a single object.
- composes objects in form of a tree structure to represent part as well as whole hierarchy.
 - the pattern creates a class that contains group of its own objects. This class provides ways to modify its group of same objects.



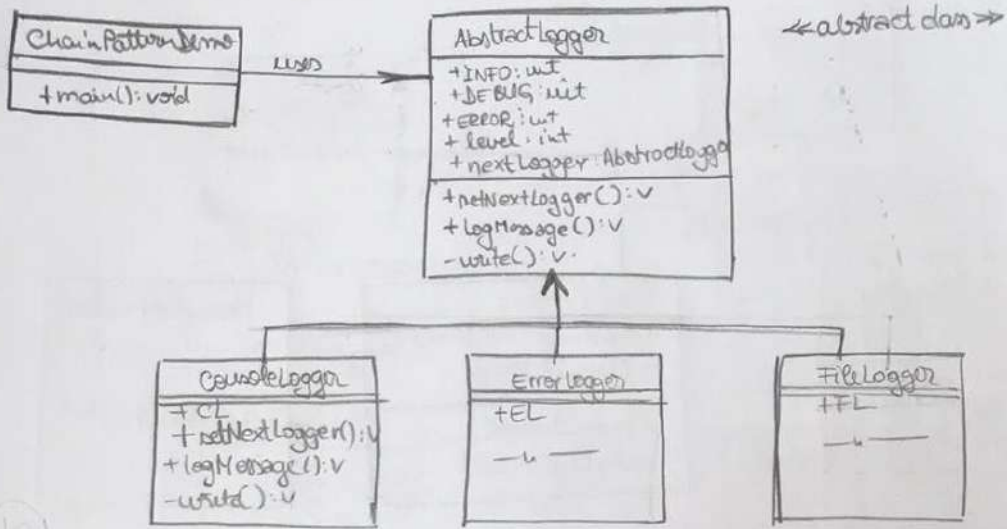
- allows a user to add functionality to an existing object without altering its structure.
- this type of dp comes under structural pattern or this pattern acts as a wrapper to existing class.
- the pattern creates a decorator class which wraps the original class and provides additional functionality keeping class methods signature intact.



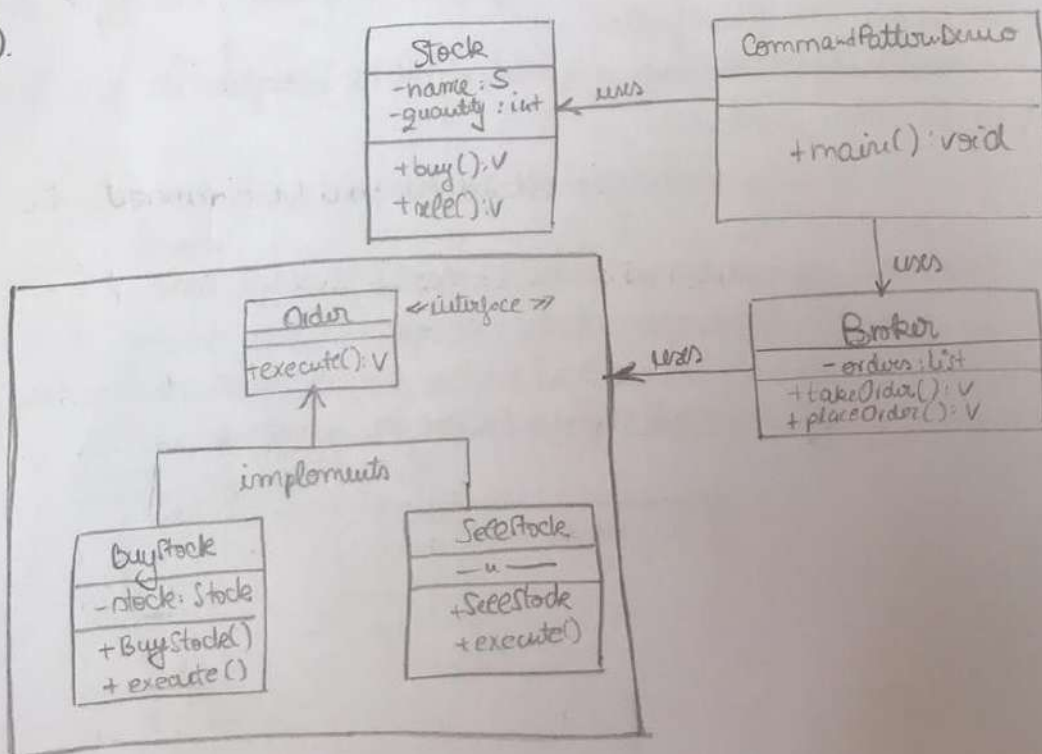
- a class represents functionality of another class. This type of dp comes under structural pattern.
- we create objects having original object to interface its functionality to outer world.

9

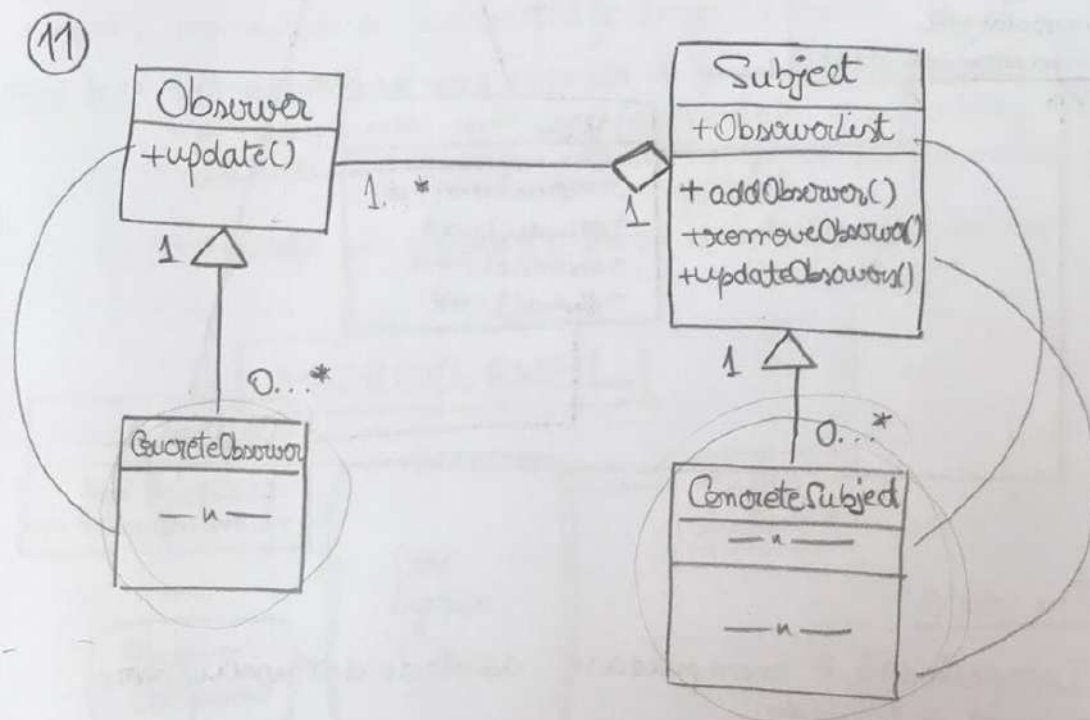
- creates a chain of responsibility pattern creates a chain of receiver objects for a request
- this pattern decouples sender and receiver of a request based on type of request ; Comes under behavioral patterns
- normally each receiver contains reference to another receiver. If one object cannot handle the request then it passes the same to the next receiver and so on.



10



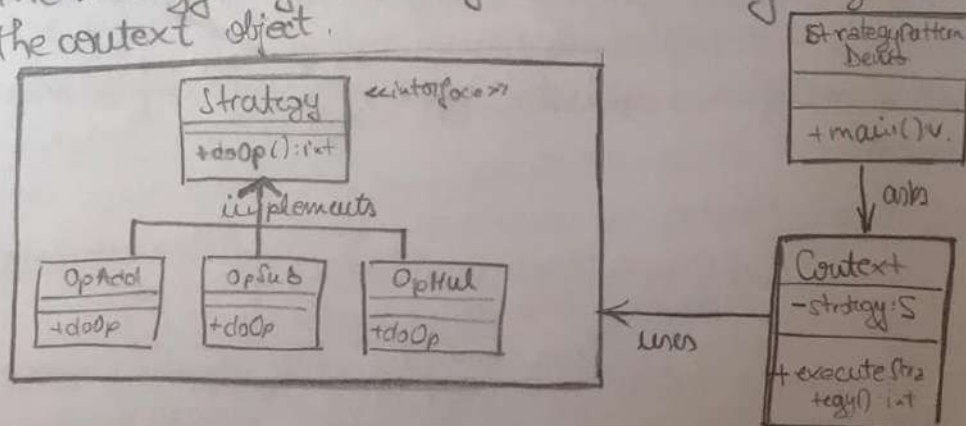
- Command pattern is a data driven design pattern and falls under behavioral pattern category.
- A request is wrapped under an object as command and passed to invoker object.
- Invoker object looks for the appropriate object which can handle this command and passes the command to the corresponding object which executes the command.



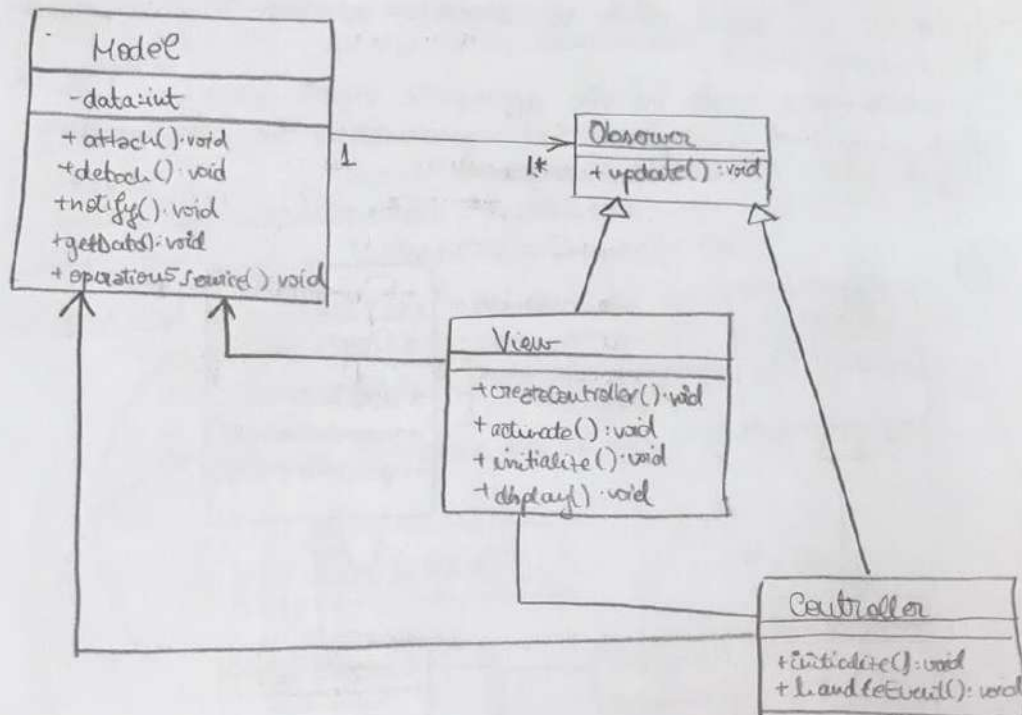
- (12) - a class behavior or its algorithm can be changed at run time.

This type of design pattern comes under behavior pattern.

- we create objects which represent various strategies and a context object whose behavior varies as per its strategy obj.
- The strategy object changes the executing algorithm of the context object.



MVC.



The Model

- Încapsulează și manipulează datele de domeniu care vor merge să fie redate.
- nu are idee cum să afișeze informațiile pe care le are, nici nu interacționează cu utilizatorul.
- Încapsulează funcționalitatea necesară manipulării, obținerii și furnizării acelor date, independent de interfața ut.

The View

- o redare vizibilă specifică a informațiilor conținute în model
- fiecare View este dependent de un model
- când se modifică modelul, toate view-urile dependente sunt actualizate

The Controller

- manipulează datele introduse de utilizator Handle requests ut Model și View.
- permite decuplarea Modelului și a View-ului, permițând View-ului să redare pur și simplu și modelul să încapsuleze pur și simplu datele

?

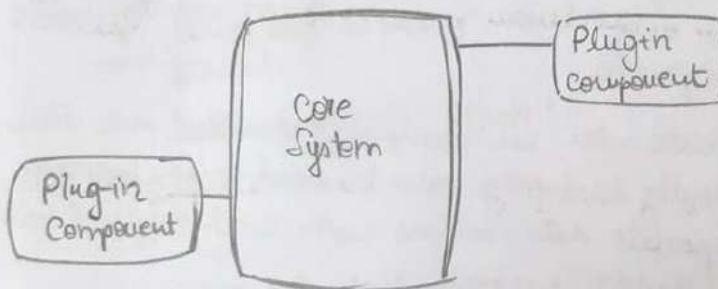
- Modelul are o listă de view-uri pe care le suportă
- fiecare view are o ref. la M-nu, precum și la O sau de restituire.
- fiecare C are o ref. la V pe care îl contribuiează, precum și la M pe care este batut V.

Is clearly a layered architecture

Avantajele l.a.e ca separă preocupările, fiecare se focusează pe un lucru. $\Rightarrow$ modularitate, testabilitate, ușor să distribuie diferite soluții, ușor de act și îmbunătățiri layer-urile separat.

- Best for:
- pt aplicații noi care necesită să fie construite repede
 - pt echipe care sunt meexpimentate și nu înțeleg alte arhitecturi până acum $\Rightarrow$ este simplă de implementat și de gestionat.
 - aplicații cu standarde stricte de întreținere și testare.

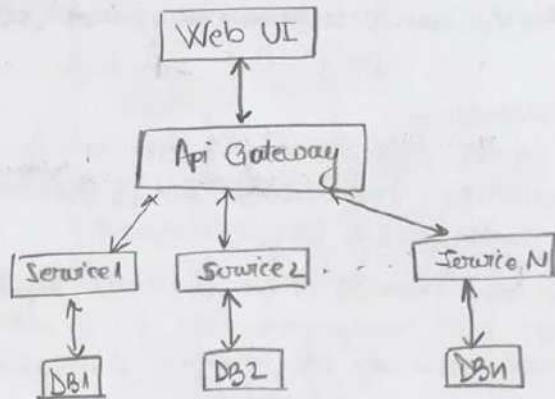
MICROKERNEL



- Avem nevoie de un sistem cu o funcționalitate minimă la care putem să adăugăm diferite funcționalități.
- Acelea sunt independente una față de cealaltă, conțin procese specializate, feature-uri adiționale și custom code.
- The Core System are nevoie să știe care P+C sunt valide și cum să ajungă la ele (plugin registry).
- The register detine informații despre fiecare P+C.
- P-modules(C) pot fi legate / conectate de core system prin: OSGi, message, web services, sau point-to-point binding

- Best for:
- multe ut de o varietate de persoane.
 - aplicații în care este clară diviziunea între rutinele de bază și regulile de ordin superior.
 - aplicații cu un set fix de rutine de bază și un set dinamic de reguli care trebuie actualizate frecvent.

MICROSERVICES



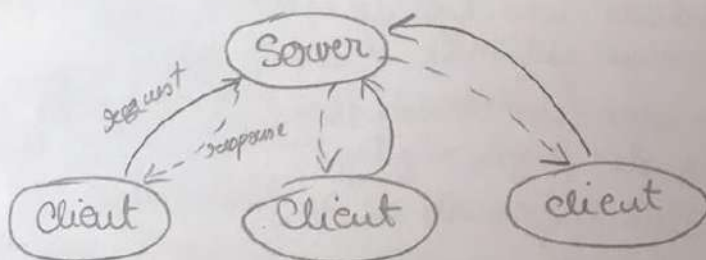
Scopul este să crești un număr de programe mici diferite și apoi să crești un program mic de fiecare dată când cineva vrea să adauge un nou feature.

Fiecare acțiune reprezintă un serviciu foarte mic a.n. jucare se ocupă de un singur lucru pe care îl face bine, fiecare fiind legată împreună.

Best for:

- website-uri cu componente mici
- rapidly developing new business and web-app.
- corporate data centers with well-defined boundaries

CLIENT SERVER



Construcția acestui pattern constă într-un server care se ocupă de funcționalitate și care poate servi mai multor clienți, respectiv clienți care consumă servicii și interacționează cu server-ul pentru a primi date.

Avantajul folosirii acestuia este în momentul în care clienții au cerințe total diferite, acestea putând fi interconectate la un server. Partitionarea task-urilor și a volumului de muncă între qumisiunile unei resurse (server) și solicitantul resursei respective (clientul). Clienții nu folosesc resurse, serverul găzduiește 1 sau mai multe programe server.

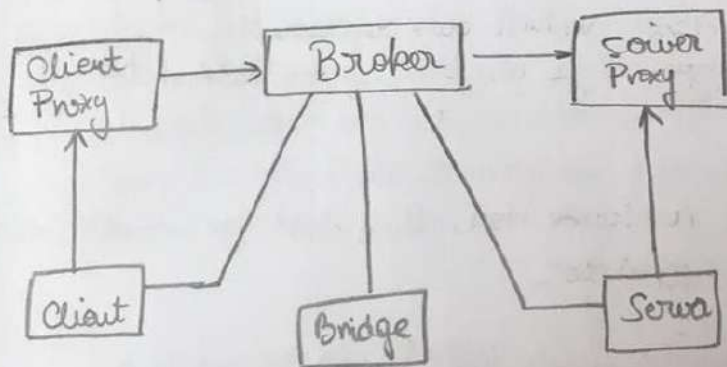
Server : provides services ; can serve multiple clients
 client : consumes services ; interacts with server to retrieve data



 Client — Internet — Server

BROKER

Mediul app e distribuit și posibil heterogen cu componente independente care cooperează.



SOLID

① class Vehicle {

public String brand;
public double price;
public int productionYear;

public String toString(String formatType) {

switch (formatType) {

case "JSON":

return toJsonFormattedString();
break;

case "XML":

return toXmlFormattedString();
break;

default:

}

S: Încalcă deoarece această clasă Vehicle care reprezintă modelarea OOP a obiectului vehicul, are pe lângă atribute și metode set & get specifice care țin funcționalități auxiliare care nu îi au locul.

O: Dacă dorim să adăugăm un nou format care trebuie să modificăm clasa Vehicle să adăugăm un case nou. Nu e dese for modification. Utilă ar fi adăugarea unei interfețe.

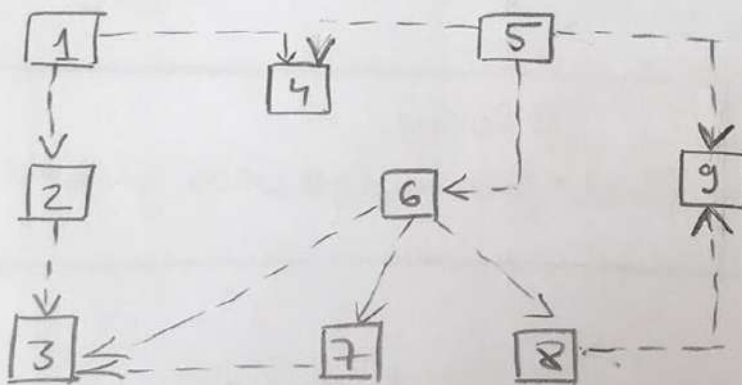
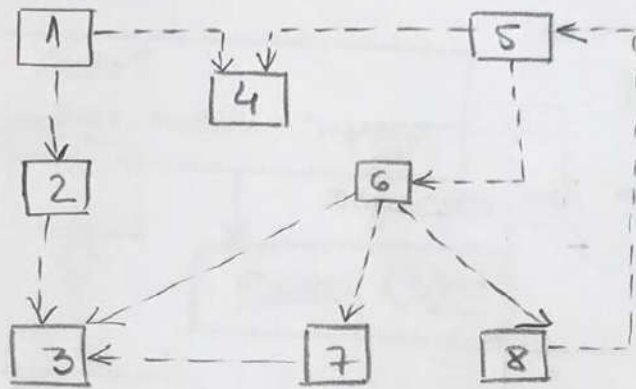
L: Încalcă deoarece o subclasă nu poate fi vădită ca și o supraclassă.

Resp. sunt declarații ambigue → funcț., nu trebuie să fie implementate obligatoriu de altă clasă (subclasă).

I: avem formatori irelevanți care în anumite condiții se pot dovedi inutile, dând probleme la mentinerea codului.

D: Abstracția de formatare nu ar trebui să depindă de Vehicle, ci de o interfață de formatare.

②



Exercitiu 4,5

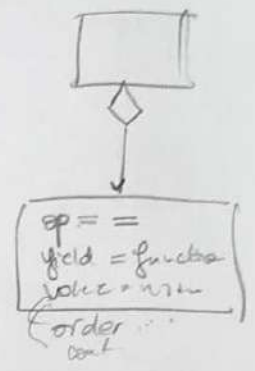
④ Query Object.

* all employees who are managers

Context
Employee.Function = "manager"

Query Object

DB Query
[Select * from employee where function = "manager"]



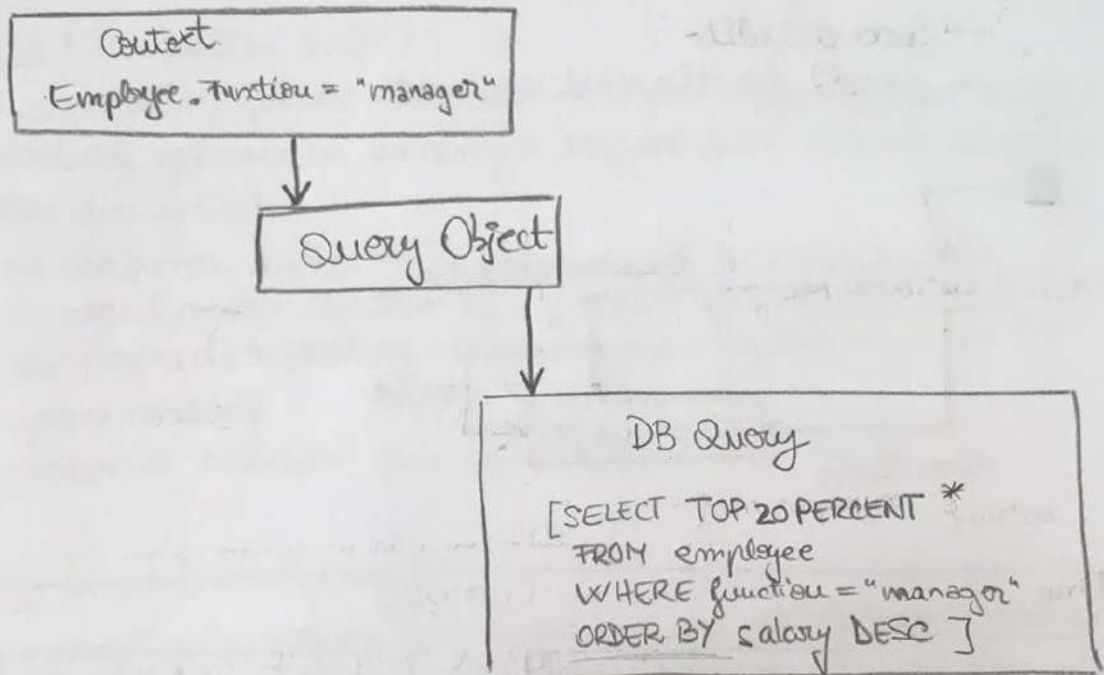
* all employees who are m. and of certain age.

Context
Employee.Function = "manager" and
Employee.Age > 50

Query Object

DB Query
[Select * from employee where function = "manager" and
age > 50]

* top 20% of these managers based on salary.

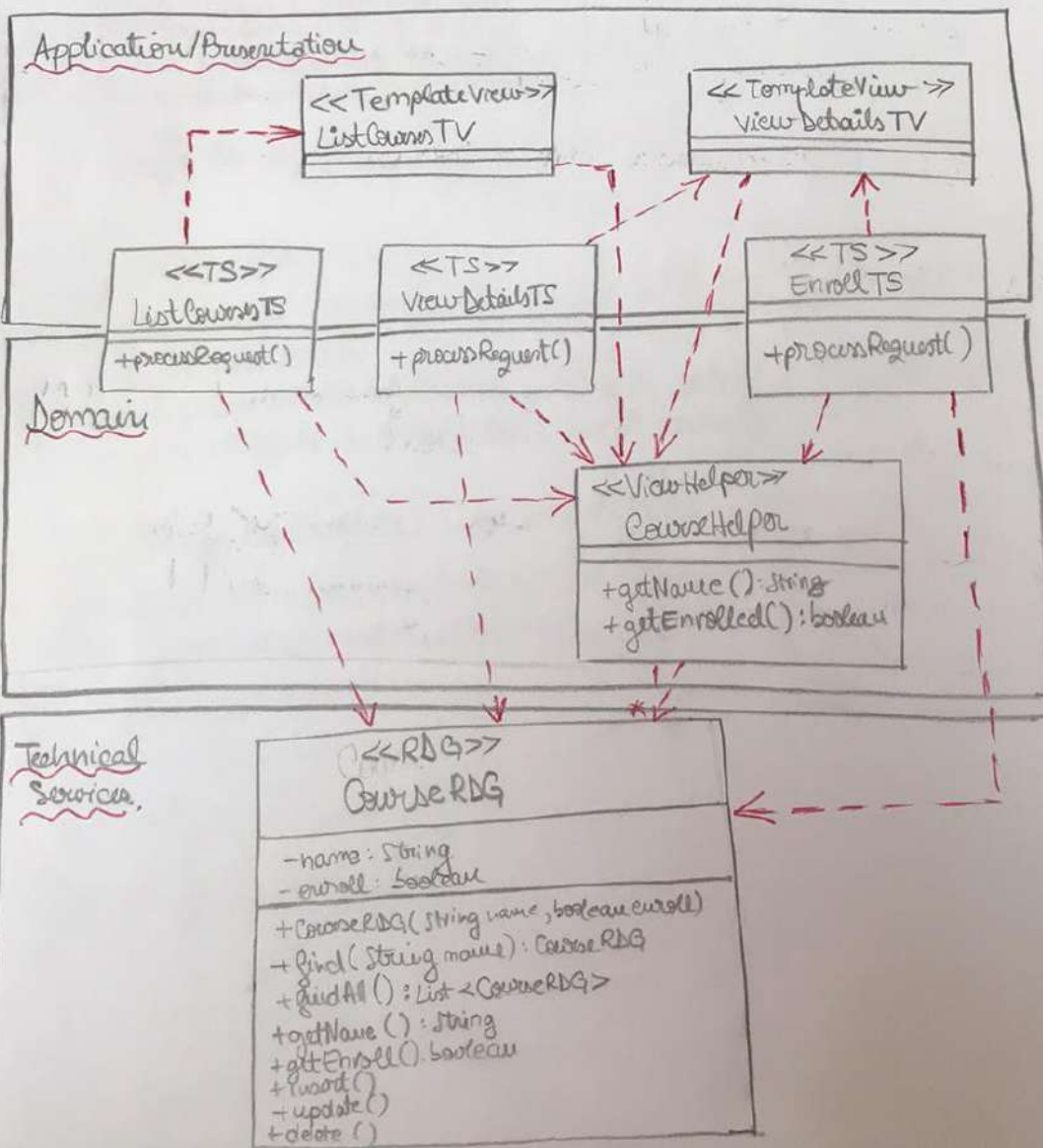
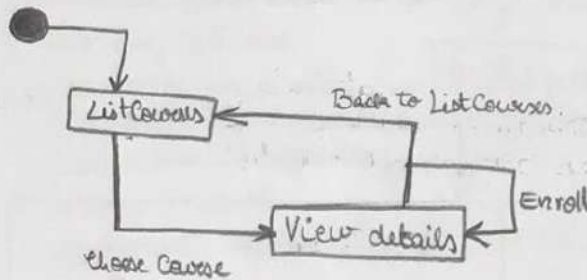


(5)

⑤

Courses:

- list of courses.
- see details
- enroll for the next summer.



În vederea rezolvării problemei propuse, am utilizat următoarea structură ierarhică : TS+RDG+TV, care reprezintă :

TS (Transaction Script)

- considerăm fiecare tranacție ca o procedură, fiecare astfel de procedură gestionează un singur request de la nivelul de prezentare din aplicație
- de asemenea, acestea fac cereri direct la baza de date (în cazul nostru, relativ la requestul primit din prezentare).
- face partea de repository, de service și de validare, toate în aceeași metodă
- procesare validări, face calcule, stocarea în BD.

RDG (Row Data Gateway)

- un obiect care returneaza un singur record din DB.
- noi creștem obiecte instanță, deci nu avem nevoie să mai folosim BD
- tipic în object-relational mapping tools (eg: hibernate)

TV (Template View)

- încorporează marcaje într-o pagină HTML statică atunci când aceasta este nouă.
- Când pagina este utilizată pentru a satisface cererile, marcajele sunt înlocuite de rezultatele unor calcule.

PC (Page Controller): - decode^{the} URL și create and invoke any model objects

- determină care view trebuie să afișeze pagina rezultat și îndreapătă informația model către acea pagină.

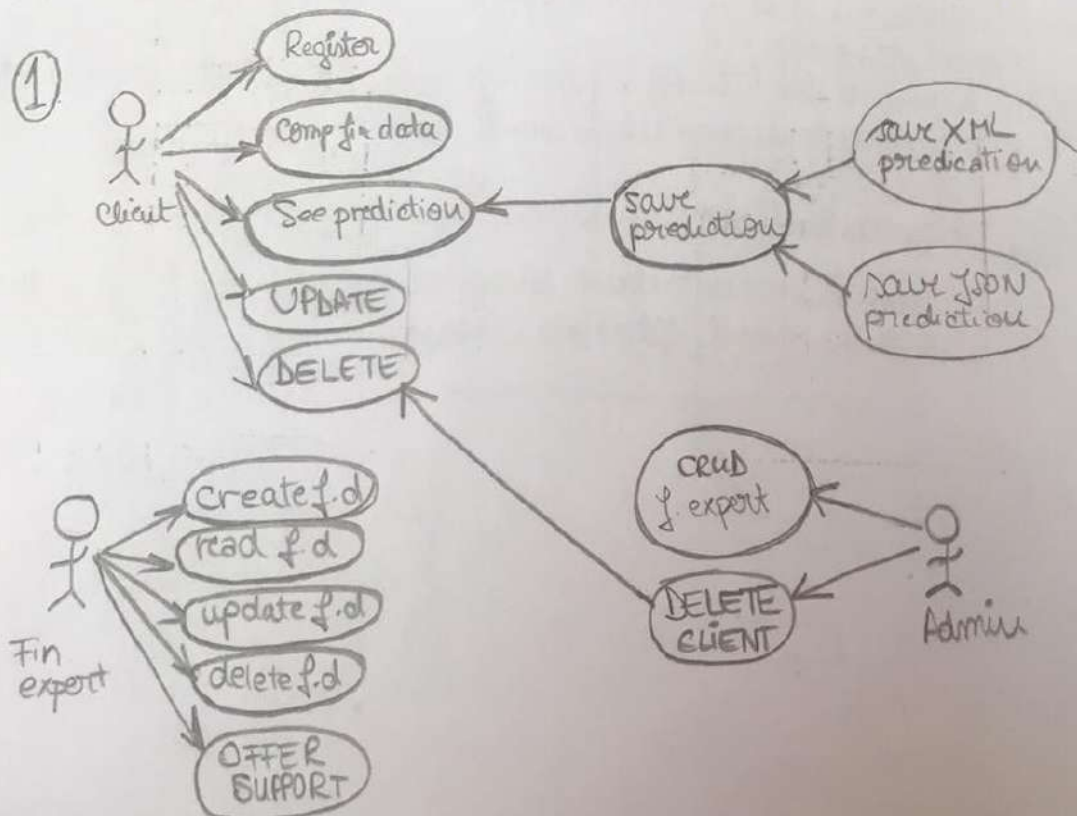
FIGO

- financial card → financial cards

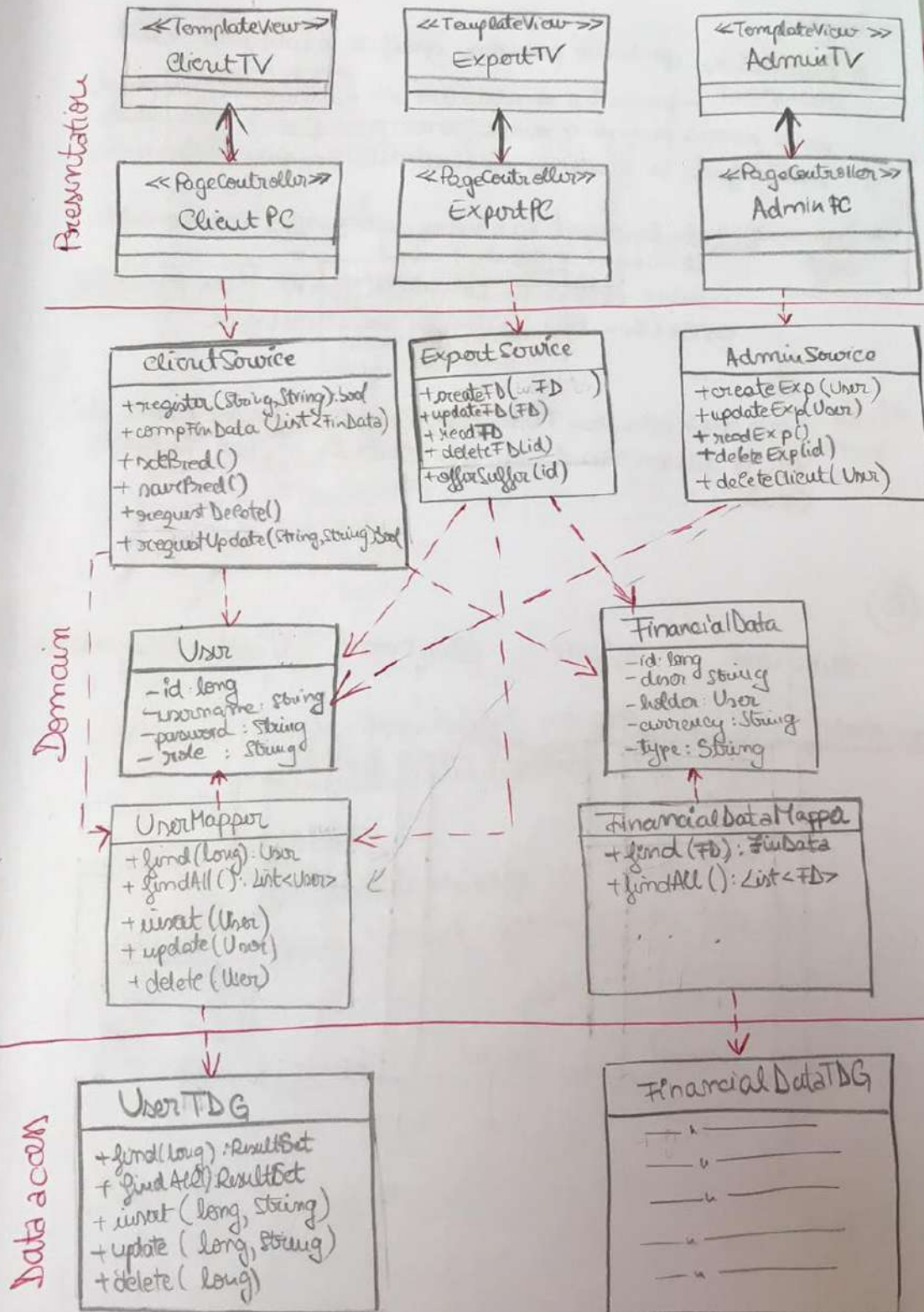
↳ id
description
card holder ID
currency
value
type

Loan (Interest, Mortgage) ^{can have linked 2 it}
Bank Asset (Deposit Acc, Saving Acc, Credit Acc etc)
Personal Asset (Car, Real Estate, Jewellery)
Expense
Insurance (Life, House, Car, etc)
Work (Employment, Freelance)
Pension (state, private)

- users: client (register, complete fin data, see pred, save pred, financial expert (CRUD fin cards & offer support), admin (CRUD on experts & delete clients at request)



② Architectural Pattern.

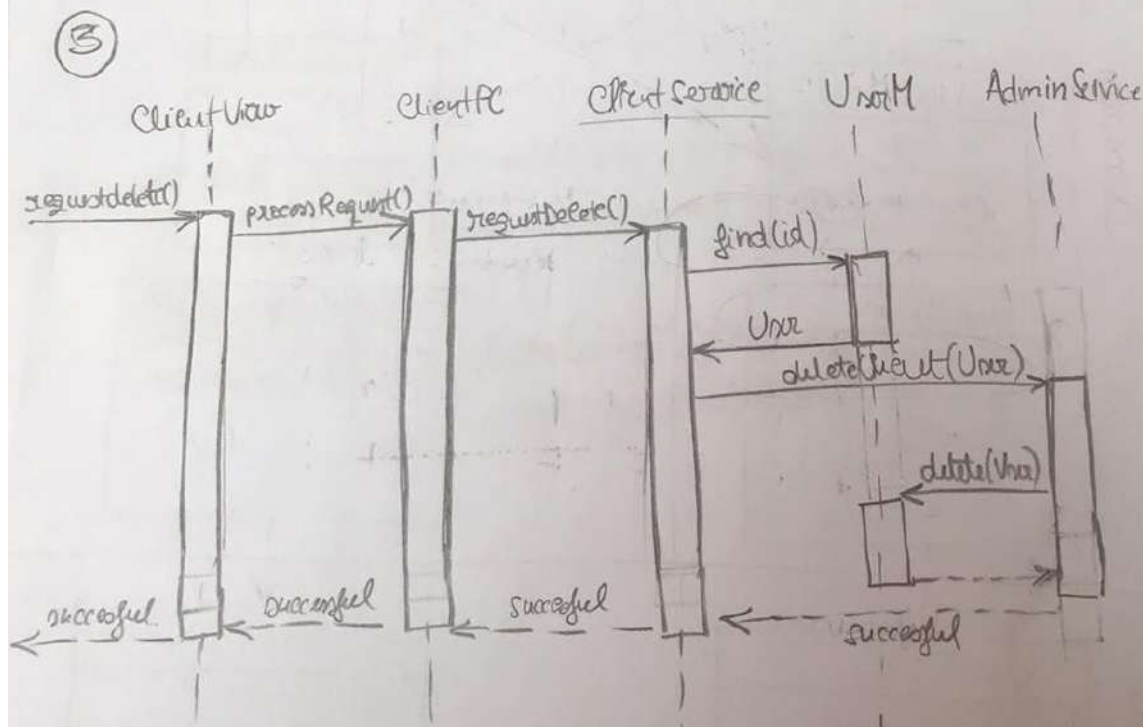


Arh. aliasă este Layer Architecture. În cazul nostru, are 3 layere:

* **Presentation** - partea de prezentare specifică o interfață cu utilizatorul
- fiecare tip de utilizator are un Controller separat pentru propria pagină și pentru fiecare pagină are un Template View diferit, în funcție de particularitățile fiecărei pagini.

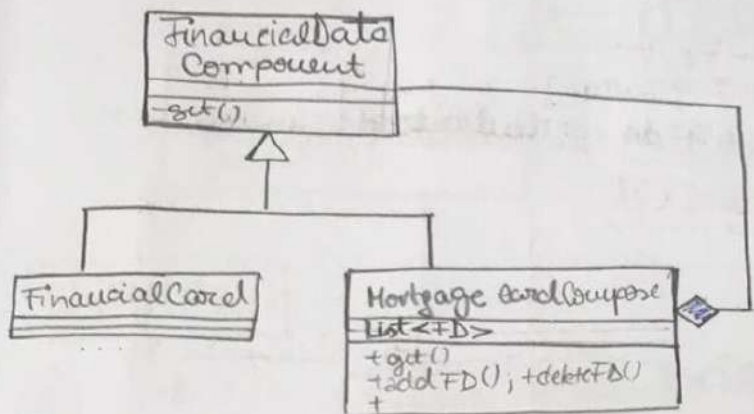
* **Domain Logic** : - este structurat ca o rețea web de clase interconectate (Domainul Modelului)
- conține clase de tip Data Mapper care țin TDB pt a accesa datele (din BD), conține de asemenea logica.

* **Data access** : - este realizată prin TDB, implementat pt. fiecare tabelă din BD, și fiecare rând; nu are atribute; doar metode CRUD.



④ UML Class Diagram.

Composite



```

(*)
public class FinancialDataComponent {
    private long id;
    private String description;
    private User holder;
    private String currency;
    private String type;
    //getters & setters
}
  
```

```

public class MortgageFinancialCard extends (*) {
    List<FinancialDataComponent> siblings;
    public void predict() {
        for (FinancialDataComp fc: siblings) {
            FactoryMethod.getPrediction(fc.type);
        }
    }
}
  
```

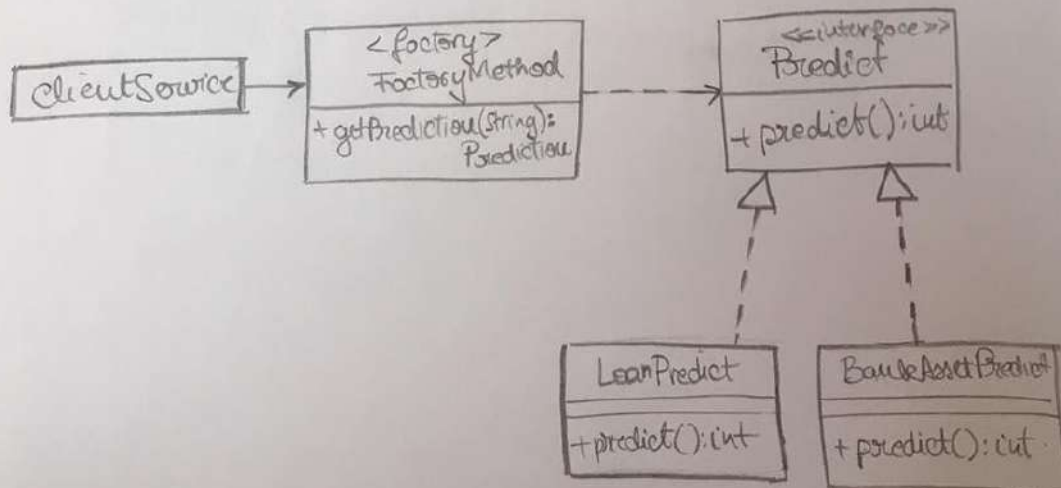
Factory Method

```
public interface Prediction {  
    public int predict();  
}
```

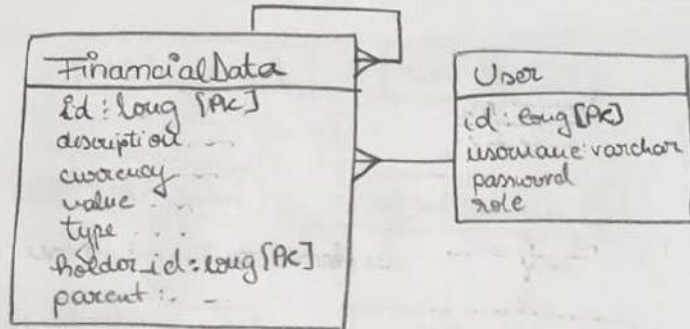
```
public class LoanPrediction implements Prediction {  
    public int predict() {  
        // ...  
    }  
}
```

```
p. c. BankAssetPred. implements — " — {  
    public int predict() {  
        // ...  
    }  
}
```

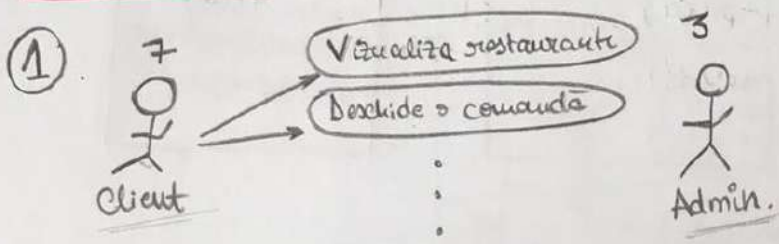
```
public class FactoryMethod {  
    public Prediction getPrediction(String type) {  
        if (type.equals("loan") return new LoanPrediction();  
        else if (—"—" ("bankasset") — " — BankAssetPrediction();  
        return null;  
    }  
}
```



5



HIPMENU



Caz de utilizare:

Use case : participate to order

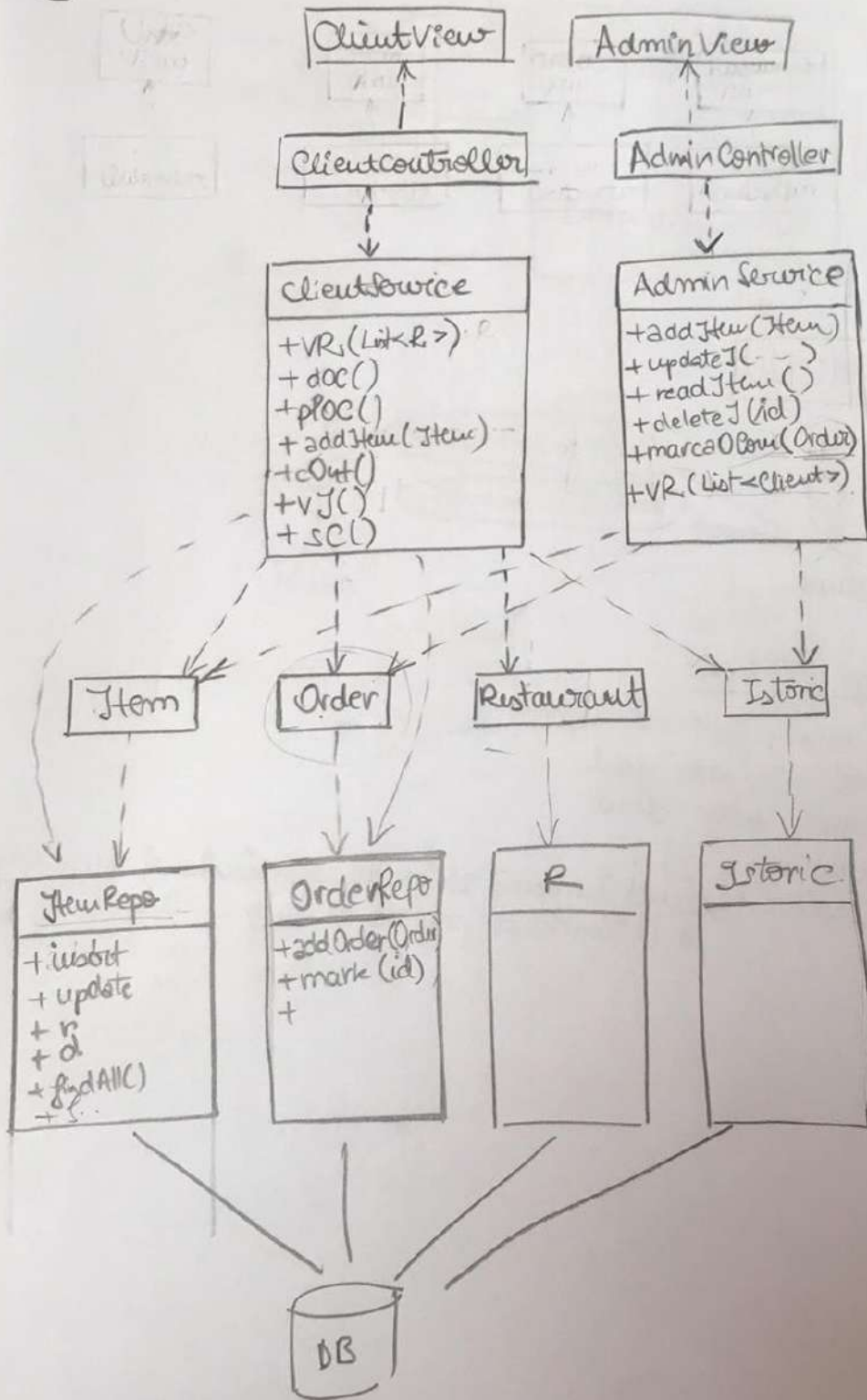
Level : user goal

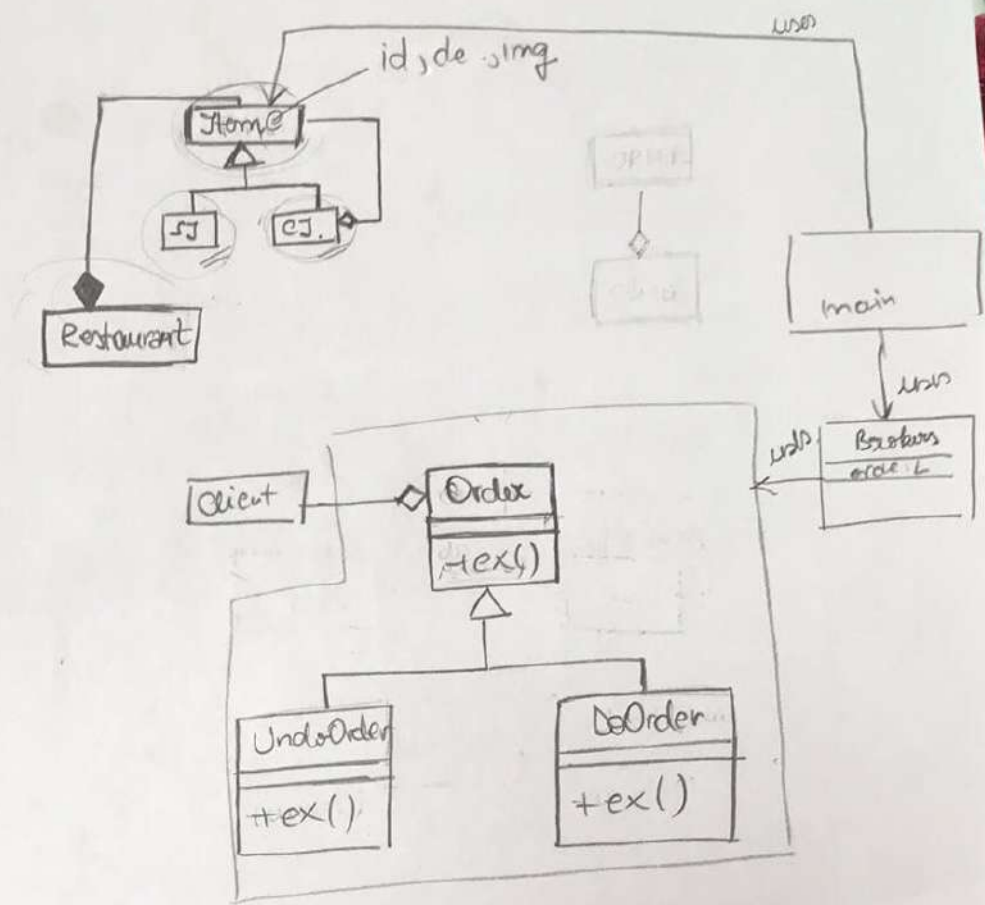
Primary actor: client

Description:

Extension : când încercă să vadă meniul și nu e logat să îl trimită pe pagina de login.

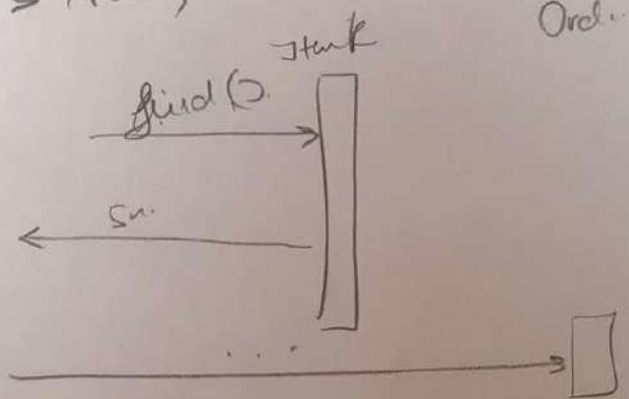
2

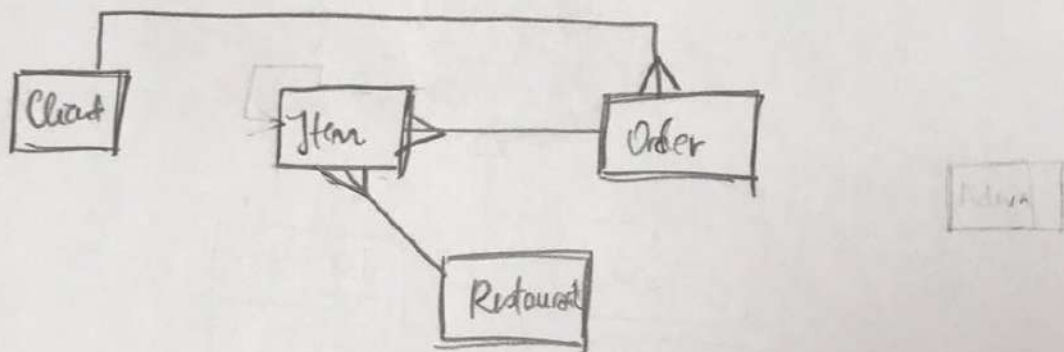
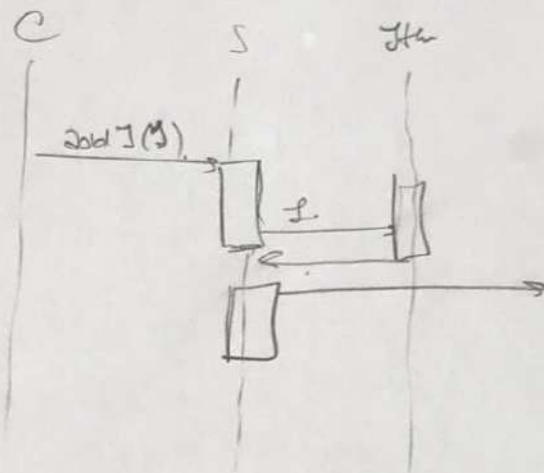


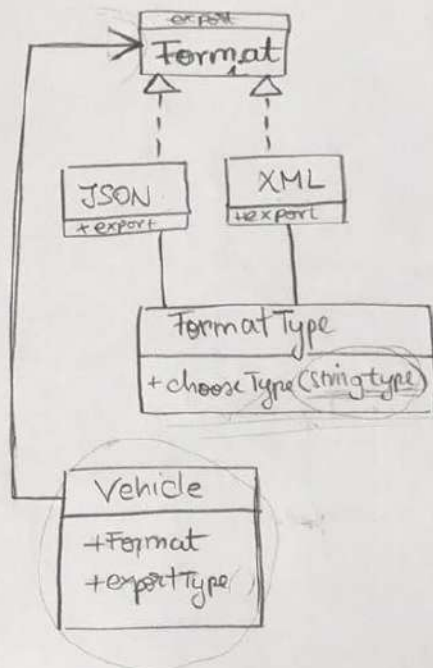


p.c. ItemC {
 — id
 — description
 — price
 }

p.c. C Item extends ItemC {
 List < ItemC > itaus;
 }







②. What class design principle is most suspected?

```
public interface IDuck {
```

```
    void swim();
```

```
}
public class Duck implements IDuck {
```

```
    public void swim()
```

```
{
    // ...
}
```

```
public class ElectricDuck implements IDuck {
```

```
    public void swim()
```

```
{
    if (!IsTurnedOn)
```

```
        throw ...;
```

```
    // swim logic ; } }
```

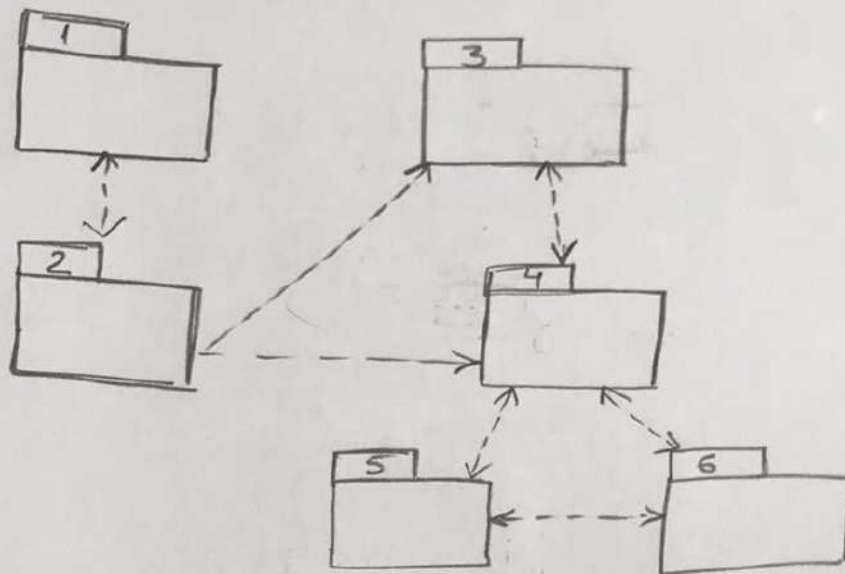
```
    // calling method
    void makeDuckSwim(Duck duck) {
```

```
        duck.swim();
```

```
}
```

PACHETE

①



⇒

